
WHITE PAPER · PORTFOLIO & AI GOVERNANCE

Every Agent Needs a Human Operating Model

What has to be true before a capable agent is allowed to act on real work in production

Marco Policani

Enterprise Portfolio · PMO · AI Operating Governance

policani.net · linkedin.com/in/marcpolicani

July 2026

EXECUTIVE SUMMARY

An agent is ready for production when one named human can answer for what it decides, when it must stop, and how many such agents that person can still own.

A capable AI agent clears its pilot, and the go-live decision lands on one leader, who must answer for everything it does next. A named human has to be able to answer seven things before the agent acts: its role, what it may decide, how it escalates and stops, what challenges its output, what a good result is, what evidence it leaves, and who owns the result. This paper puts them on one page, the Agent Operating-Model Canvas.

- 01** Before an agent ships, one named person must be able to account for its decisions. If no name attaches, it is not ready.
- 02** Write down what the agent may decide, what needs sign-off, and what it must pass up. Silence gives it the widest authority anyone can read into it.
- 03** The amount of human supervision an organization can supply, not model quality, is the likely limit on how many agents it can run safely. Fund that oversight as a standing cost, and make a completed canvas the gate before an agent is funded.

CONTENTS

The go-live decision	One person can only own so many agents
Why your current controls do not fit agents	Three agents with a human owner
The vendor is not the owner	What this changes for the portfolio
A work-design test, not a technical one	Two things the evidence does not settle
The Agent Operating-Model Canvas	What to count instead of deployments
How each gap shows up in the work	Notes and sources

The go-live decision

A capable AI agent, software that takes actions on its own, clears its pilot. The decision to put it into production lands on one leader's

desk. Approving it means assigning a license, pointing the agent at a live workflow, and letting it act on real customers and real data. At that moment the leader has no settled answer to a plain question. The agent makes a judgment call, escalates the wrong case, or returns an output that looks finished and is wrong. Who in the organization owns that result?

My claim is narrow and testable. An agent belongs in production only after it has an operating model. Seven things have to be in place: a defined job, written rules for what it may decide alone, a way to escalate and a way to stop it, something that challenges its output, a standard for a good result, a duty to leave evidence, and one accountable human owner. Without those seven elements, the agent is running work that no one in the organization can answer for, however capable the model behind it.

Enthusiasm for agents is not in short supply. McKinsey reported in 2025 that 88 percent of respondents said they used AI regularly in at least one business function. Only 39 percent reported an effect on earnings at the enterprise level.[1] Gartner goes further on agents. It projects that more than 40 percent of agentic AI projects will be canceled by the end of 2027. The reasons it cites are rising costs, unclear business value, and weak risk controls.[2] Gartner does name immature models as a factor. Its headline reasons, though, and its warning that many agentic projects are driven by hype and misapplied, point past the model to how agents are chosen and governed.

This is an argument for governing agents, and governing is easy to mistake for slowing down. A committee that an agent program has to satisfy is just delay no one owns. Real governance does the opposite. It lets an organization move fast and still answer for the result, however many agents it runs. The thing that makes that possible is the operating model, and it fits on one page.

Why your current controls do not fit agents

Companies built their operating models to keep control. Those models assume steady processes, clean handoffs, and predictable results. A person sits at each decision point and makes the decision that matters. Agents do not work that way. They decide on their own, adapt as they run, and produce results that cannot always be anticipated. Bolt an agent onto a model built for human workers and, as Deloitte puts it, you are fitting a jet engine to a bicycle.[3]

The deeper problem is a category problem. An agent acts like a worker but is funded like technology. Deloitte finds that this mismatch opens gaps exactly where accountability should sit. The gaps are who decides what, who carries the risk and liability, who assures quality, and who answers for performance.[3] A company knows how to govern a software purchase. It knows how to govern an employee. It has almost no settled practice for something it buys like the first and that behaves like the second.

There is evidence the gap is going unaddressed. Deloitte's State of AI in the Enterprise 2026 survey found that 84 percent of companies had not redesigned jobs to fit AI, even though they expected a lot from automation.[3] Redesigning the job is what produces decision limits, escalation paths, and ownership. Skipping it does not make the questions go away. It saves them for the moment an agent does something no one authorized and no one can explain.

Deloitte warns that layering agents onto broken processes does not fix the processes. It amplifies the problems.[3] Give an agent an ambiguous workflow and it will resolve the ambiguity faster and more confidently than a person would. It will also do it in far more cases, which makes the error harder to catch. The instinct to move quickly is not wrong. The mistake is treating deployment as the finish line. It is the point where the governance question becomes real.

The vendor is not the owner

Because companies buy agents like technology, they are tempted to treat accountability as part of the purchase. It is not. A vendor can warrant that a model performs to a specification. It cannot own the consequences of a decision the agent makes inside your workflow, against your data, on behalf of your customer. Accountability lives in the operating model you build around the product. You cannot hand it to the company that sold you the software.

That is why the owner has to be a named person inside the company, not a clause in a contract. Say the agent misclassifies a case, releases a wrong figure, or escalates something it should have handled. The regulator, the customer, or the executive does not ask which vendor supplied the model. They ask who here was answerable for letting the agent act. If the honest answer is a line in a purchase order, the agent never had an operating model — only a license.

The test is simple, and it is worth applying before any agent goes live. Name the person who would stand up in a review and account for what the agent decided. If

that name does not exist, or if it is really a group that adds up to no one, the deployment is premature. That holds however capable the model is and however confident the vendor is.

A work-design test, not a technical one

An agent has an operating model when one named human can answer seven questions about it before it goes live. If any answer is missing, the agent is not ready for production, however capable the underlying model is.

This is a work-design test, not a technical one. Accuracy, latency, and cost matter, but they are not what fails in the field. What fails is the missing owner when a judgment goes wrong. What fails is the missing escalation path when the agent hits a case it should not decide. What fails is the missing evidence trail when someone asks why a decision was made. Those are governance problems, and one page can hold the answers to all of them.

The Agent Operating-Model Canvas

The canvas has seven elements. Anyone who has run a program or a portfolio will recognize every one of them. The discipline is applying them to a non-human worker before it starts, rather than after it has caused a problem.

EXHIBIT 1

Who answers for what**1. Role and scope**

What the agent is for, and what it is not for.

2. What it may decide

Decide alone, get sign-off, or pass it up. In writing.

3. Escalation and stop

A path for cases it should not handle, and a halt that already exists.

4. What challenges it

Say what must challenge the output before anyone trusts it.

5. A good result

Define what good looks like before the output arrives.

6. Evidence it leaves

Keep a trail a human can follow.

7. Who owns the result

One named person answers for it. Not a committee, not the vendor.

1. Role and scope

Write down what the agent is for and, just as important, what it is not for. Scope is the first control because it sets the limits of every other one. An agent that drafts a document and flags exceptions is a different governance problem from one that sends the document to a customer. Scope creep is the most common way an agent ends up making decisions no one designed it to make. Writing the limit down is what makes creep visible before it becomes an incident.

2. What it may decide, and what it may not

Decide in advance what the agent may decide on its own, what needs a human sign-off, and what it must pass up the line. Deloitte splits it the same three ways and adds an important rider: set expectations for the evidence the agent must produce behind each decision.[3] Most agent programs say nothing here, and silence gives the agent the widest authority anyone can read into it. A written rule — it may reclassify but not approve, recommend but not release — turns a hidden risk into a control you can see.

3. How it escalates and stops

Set the path for the cases the agent should not handle, and the switch that stops it when something is wrong. Deloitte's guidance here is concrete. Build stop mechanisms, say who resolves exceptions, and decide where slowing the work down is worth it, so errors do not spread before a human notices.[3] Speed is the point of an agent. Speed with nothing to stop it is also how one bad assumption becomes a thousand bad outputs. A stop mechanism is the difference between an incident and a catastrophe, and it has to exist before it is needed. Nobody builds one well in the middle of a failure.

4. What has to challenge it

Decide which outputs have to be challenged before anyone trusts them. An agent that only confirms is more dangerous than one that sometimes objects, because confident wrong answers are the expensive ones. In mature agentic systems this becomes structural. McKinsey describes critic agents that challenge outputs, guardrail agents that enforce policy, and compliance agents that watch regulation, with every action able to be logged and explained in real time.[4] Most organizations are not there yet, and they do not need to be. The rule can be light. A second agent, a checklist, or a human reviewer — anything other than accepting an output because it arrived fast and looked complete. The protocol answers one standing question. Before anyone trusts this output, what had to disagree with it and be overruled?

5. What a good result looks like

Define what a good result looks like before the agent produces one. Written standards are the difference between reviewing an agent's work and merely receiving it. Without them, review collapses into a check for plausibility, which is exactly the check a fluent model defeats best. Keep the standards specific to the workflow. Name the fields that must be correct, the sources that must be cited, the thresholds that must not be crossed, and the point at which the output is rejected rather than edited. Written standards also let someone else do the review, because a second person can apply them without guessing what the first reviewer had in mind.

6. What evidence it leaves

Keep a trail that lets a human work out what the agent did and why. Deloitte's list is a workable minimum: log and review what the agent does, audit its behavior, write down the reasons and the human interactions, and track disagreements.[3]

Public standards give that duty a concrete form. The NIST AI Risk Management Framework organizes oversight around four functions: govern, map, measure, and manage. Together they form a cycle for understanding context, judging performance and risk, and deciding how a system should be used over time.[5] Its Generative AI Profile goes further. It treats risk as a lifecycle concern across design, deployment, operation, and decommissioning, and it calls for continuous monitoring, structured feedback, and incident response.[6] Keeping the evidence is what turns those documents into daily practice.

7. Who owns the result

Name one person who owns what the agent produces. Not a committee, and not the vendor. McKinsey describes humans moving above the loop: setting policy, watching for outliers, and adjusting how involved they are rather than checking every action line by line.[4] Above the loop still means inside it. The owner answers for the agent's decisions, can hit the stop switch, and is accountable for the evidence trail being real. If no name can be attached to an agent, the agent does not have an operating model, and it should not be in production.

How each gap shows up in the work

Most agent failures show up as a symptom in the work long before anyone calls them a governance problem. The trick is to read the symptom back to the missing element, because each gap needs a different fix.

Missing element	How the gap shows up in the work	What it costs
Role and scope	The agent is quietly doing tasks no one assigned it	Unowned decisions accumulate outside any review
What it may decide	No one can say whether the agent was allowed to make the decision	Authority defaults to the widest reading; risk is invisible
How it escalates and stops	A bad output is discovered only after it has propagated	A single wrong assumption scales into many wrong actions
What has to challenge it	Confident answers are accepted because they look complete	Fluent errors pass review; quality erodes silently
What a good result looks like	Review becomes a plausibility check, not a real one	The organization cannot tell good output from convincing output
What evidence it leaves	No one can reconstruct why a decision was made	Failures cannot be diagnosed, audited, or learned from
Who owns the result	When something breaks, the answer is that the system did it	Accountability evaporates; the same failure repeats

The table pairs each missing element with how the gap tends to show up and what it costs. The right column is the argument in miniature. Every entry is a governance cost, and a better model fixes none of them. All of them are fixed by deciding in advance the seven things a human owner must be able to answer.

One person can only own so many agents

The canvas reads like a checklist you fill in once per agent. That invites an obvious next move: fill it in again and keep adding agents. The seventh element breaks that plan. Every agent needs one named human owner, and no human can own an unlimited number. Each new agent draws on the same finite pool of attention, review, and judgment. So the canvas that governs one agent also caps how many an

organization can run. That forces a question most leaders underestimate: how many agents can one person actually own?

EXHIBIT 2

More agents without more exposure

The instinct

Add agents until the models stop improving.

The reality

Human oversight runs out first. Unwatched agents add exposure, not output.

The planning number

Span of supervision: how many agents one owner can genuinely answer for.

The funding move

Budget oversight as a standing cost. A completed canvas is the funding gate.

Oversight is finite, and it is becoming the real limit on how far an agent program can grow. McKinsey reports that a team of two to five people can already supervise an agent factory of 50 to 100 specialized agents running a process end to end.[4] That ratio is also a warning. The same report concludes that how far agents spread will be capped by how much oversight people can provide, which makes governance itself a potential bottleneck to productivity.[4] The limit predates AI. In a simulation of one operator supervising several semi-autonomous drones, Cummings and Mitchell found that operators could reach cognitive saturation, the point where they can take in no more. They also found that adding automated decision support brought failures of its own, including fixation, staying locked on one thing, and automation bias, trusting the machine too much.[7]

The instinct is to treat model capability as the limit and add agents until the models stop improving. In practice, human oversight runs out first. An organization that adds agents without adding — or redesigning — the oversight to hold them will not grow its output. It will grow its exposure. That is an inference from the evidence rather than a measured law, and it should be treated as one. Still, McKinsey's projection and the supervisory-control study point the same way: how much people can supervise is what caps safe growth.

There is a partial escape, and it is already visible. Technology can extend oversight itself — what Deloitte describes as agents guarding other agents that are then

guarded by humans.[3] Critic, guardrail, and compliance agents stretch the reach of a small human team. The NIST Generative AI Profile supplies actions that fit this kind of oversight. It names continuous monitoring, red-teaming (deliberate attempts to break the system), independent evaluation, and incident and feedback loops used in deployment and decommissioning decisions.[6]

Layered oversight, though, raises the ceiling without removing it. Final accountability still rests with a named person. The number to plan with is the span of supervision: how many agents one human owner can genuinely answer for. It belongs in the portfolio conversation next to cost and value.

Three agents with a human owner

These are typical situations, not claims of broad enterprise AI ownership. In portfolio and PMO work the lesson recurs. An agent is most useful when it sits on a defined, repeating workflow with a named human owner and an evidence trail. It is most dangerous when someone hands it a workflow no one has redesigned.

In one situation from portfolio governance, an agent prepares the evidence for a decision. It breaks down plan data, points out inconsistencies, and assembles the status that a leadership meeting would otherwise spend its time rebuilding. The operating model matters more than what the agent can do. Its job is preparation, not decision. It may recommend, not approve. Its standard is the set of fields and dependencies a reviewer has to be able to trust. A named owner also signs off before anything reaches the meeting. The value is real, but the governance around the agent creates it, not the model alone.

In a second situation, an agent proposes reclassifications across a large set of initiatives. Left unsupervised, it would impose a filing scheme no one agreed to. Inside an operating model it recommends, a product owner confirms or amends, every change is logged, and the disagreements are tracked. The agent widens coverage. The human keeps the judgment. That split — coverage from the agent, judgment from the human — is the split that survives contact with real work.

A third situation is intake triage. An agent reads incoming requests and routes them to the right queue. It looks low-risk, which is exactly why it is worth governing. A single mis-route is quiet, and a steady run of them stays invisible until a backlog shows up somewhere no one is watching. The operating model that makes it safe is unglamorous. The agent routes. Low-confidence cases go up the line instead of being guessed. The routing decisions are logged so a weekly review can catch drift.

One owner is also accountable for how well the routing works. The agents most likely to escape governance are the ones that look too minor to need it.

What this changes for the portfolio

Taken together, the canvas and the oversight ceiling change how an agent program should be funded. An agent's cost does not end at the license. It carries a standing oversight cost — the owner's attention, the review time, the evidence handling — for as long as it runs. A portfolio that funds agents without funding their oversight understates the cost and overstates the capacity it has. It finds the gap at the worst time, when too many agents are live for too few owners to answer for.

Make the operating model a funding gate, a required checkpoint, not an afterthought. Before anyone approves an agent, the canvas is complete. It has an owner, written limits on what the agent may decide, a stop mechanism, a standard for a good result, and a budgeted share of someone's oversight capacity. An agent that cannot clear that gate goes back to have its operating model designed, which is usually where the real work was hiding. That keeps the portfolio honest about what it can actually supervise, and it keeps the number of live agents tied to the number of humans who can answer for them.

Two things the evidence does not settle

Two things in this argument are not settled by the current evidence, and naming them is more useful than papering over them. First, the span of supervision is not a known constant. McKinsey's 50 to 100 agents per small team is an early observation reported by McKinsey, not a benchmark that carries over to every workflow, risk level, or regulatory setting. In a high-liability process the safe number may be far lower.[4] Treat the span of supervision as a number to measure in your own setting and revise as evidence accumulates.

Second, no one has yet put a good number on what oversight returns. The sources cited here point to oversight capacity as a cap on safe growth. They do not tell a leader precisely how much oversight a given set of agents needs to stay safe, or what happens to the failure rate when that oversight is thin. Until that evidence matures, the defensible position is conservative. Budget oversight as a real cost. Measure the span of supervision you can actually sustain. Let the oversight you have set the number of live agents you run.

What to count instead of deployments

The temptation with agents is to measure progress by deployment: how many are running, how much they touch, how fast they move. That number is easy to grow and easy to mistake for value. A different measure predicts whether the value survives: whether a named human can still answer all seven questions for every agent that is live.

Come back to the leader with the go-live decision still on the desk. The honest go/no-go turns on one thing. For this agent and the ones already running, can a named person say what it may decide, when it must stop, and whether there is still enough human ownership to go around? Give every agent a role, an owner, written limits on what it may decide, a challenge protocol, a standard for a good result, an evidence duty, and a way to be stopped. Then the organization has capability it can answer for. Skip that and it has built something it will later have to explain, usually at the worst possible time. Build the operating model first. Then let the agent work. The model was never the real limit. The number of agents you can safely run is the number your people can still answer for.

Notes and sources

1. Alex Singla, Alexander Sukharevsky, Bryce Hall, Lareina Yee, Michael Chui, and Tara Balakrishnan, "The state of AI in 2025: Agents, innovation, and transformation," McKinsey & Company, November 5, 2025. Verified July 10, 2026.
<https://www.mckinsey.com/capabilities/quantumblack/our-insights/the-state-of-ai-2025>
2. Gartner, "Gartner Predicts Over 40% of Agentic AI Projects Will Be Canceled by End of 2027," press release, June 25, 2025. Verified July 10, 2026.
<https://www.gartner.com/en/newsroom/press-releases/2025-06-25-gartner-predicts-over-40-percent-of-agentic-ai-projects-will-be-canceled-by-end-of-2027>
3. David Mallon, Brad Kreit, and Natasha Buckley, "Rethinking operating models for humans with agents," Deloitte Insights, April 2, 2026. Verified July 10, 2026.
<https://www.deloitte.com/us/en/insights/topics/talent/operating-models-for-humans-ai-agents.html>
4. Alexander Sukharevsky, Alexis Krivkovich, Arne Gast, Arsen Storozhev, Dana Maor, Deepak Mahadevan, Lari Hämäläinen, and Sandra Durth, "The agentic organization: Contours of the next paradigm for the AI era," McKinsey & Company, September 26,

2025. Verified July 10, 2026. <https://www.mckinsey.com/capabilities/people-and-organizational-performance/our-insights/the-agentic-organization-contours-of-the-next-paradigm-for-the-ai-era>

5. National Institute of Standards and Technology, "AI Risk Management Framework Core," excerpt from AI RMF 1.0, 2023. Verified July 10, 2026. <https://airc.nist.gov/airmf-resources/airmf/5-sec-core/>

6. National Institute of Standards and Technology, "Artificial Intelligence Risk Management Framework: Generative Artificial Intelligence Profile," NIST AI 600-1, July 2024. Verified July 10, 2026. <https://doi.org/10.6028/NIST.AI.600-1>

7. M. L. Cummings and P. J. Mitchell, "Operator Scheduling Strategies in Supervisory Control of Multiple UAVs," Aerospace Science and Technology 11 (2007): 339–348. Verified September 15, 2026. https://dspace.mit.edu/bitstream/handle/1721.1/90289/CummingsMitchell_AST06.pdf

About the author

Marco Policani is an enterprise portfolio, PMO, and AI operating-governance leader. He builds portfolio governance systems where AI carries the analytical load and named humans own every decision — an approach documented across case studies, walkthroughs, and working governance guides on his portfolio site.

Portfolio & governance library: policani.net/governance · Full portfolio: policani.net · LinkedIn: linkedin.com/in/marcpolicani

This white paper may be shared freely with attribution. © 2026 Marco Policani.